

The path-aware SPO portfolio tree

What the pipeline in `main.py` and `run.py` does, precisely

18 September 2026

Code state: commit `36b2ffc` plus the uncommitted changes of `_best_split` (parent refit, largest regret reduction) and of `tests/test_tree.py` (two new tests); 35 tests pass.

At a glance.

- **Pipeline.** Daily closes and your daily features $\rightarrow$ one row per weekly decision $\rightarrow$ a decision tree whose leaves hold a *predicted-return vector* $\rightarrow$ an exact portfolio optimizer with fees and a covariance penalty $\rightarrow$ weights. A replay trades the tree week by week, with drifting holdings and fees, and gives the Sharpe ratio.
- **What is learned.** Leaf values minimise the *decision regret* (SPO loss) of the portfolios they induce. Candidate splits are tried in decreasing order of regret reduction, and the first one that also raises the Sharpe ratio of the tree’s own replayed path is accepted.
- **Status.** The code does what it claims: 35 unit tests, 20 planted bugs all caught, and an audit (Section 7). On synthetic crypto-like data, extra leaves always improve the training figures; on the test year their gain is small and hard to tell from luck, except when the signal is strong and the history long. *No result in this report comes from real BTC or ETH data.*

Overview and notation

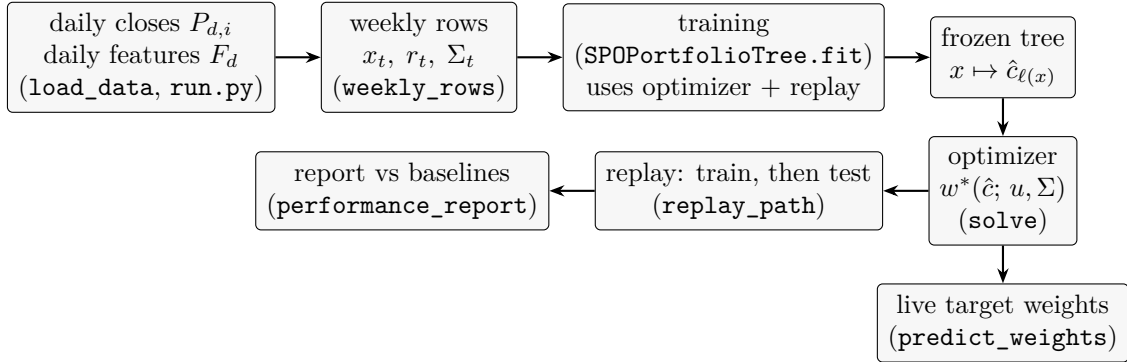


Figure 1: The pipeline. `run.py` executes it end to end once `load_data()` is filled in.

Abbreviations and sources. *SPO*: Smart Predict-then-Optimize, training a predictor on the quality of the decision it induces. *SPOT*: Elmachet, Liang and McNellis, “Decision Trees for Decision-Making under the Predict-then-Optimize Framework”. *DIR*: Wang and Hasuik, “Decision-Induced Ranking Explains Prediction Inflation and Excessive Turnover in SPO-Based Portfolio Optimization”. *W–H SPO article*: Wang and Hasuik, “Smart Predict-then-Optimize Paradigm for Portfolio Optimization in Real Markets”. *DSPO*: Zhong et al., “DSPO: An End-to-End Framework for Direct Sorted Portfolio Construction”. “The Wang–Hasuik articles” means DIR and the W–H SPO article.

Symbol	Meaning	In the code
n, p	number of assets ($2 \leq n \leq 4$, numbered from 0 as in the code) and of features	<code>n_assets</code> , <code>X.shape[1]</code>
$P_{d,i}, F_d \in \mathbb{R}^p$	close of asset i and feature vector on calendar day d	<code>closes</code> , <code>daily_features</code>
$t = 1, \dots, T; d_t$	weekly decisions, and the day of decision t	rows of <code>WeeklyRows</code>
x_t, r_t, Σ_t	features, next-week simple returns, weekly covariance estimate	<code>X</code> , <code>R</code> , <code>Sigma</code>
u_t, w_t	weights just before trading (holdings) and after trading	<code>holdings</code> , <code>weights</code>
$f \in \mathbb{R}_{\geq 0}^n, \lambda > 0, \bar{w}$	fee per traded notional (one value per asset), risk aversion, cap per asset	<code>fee_rate</code> , <code>risk_aversion</code> , <code>max_weight</code>
$c, \hat{c}_\ell \in [-b, b]^n$	a predicted-return vector; the one stored in leaf ℓ	<code>prediction</code> , <code>prediction_bound</code>
I_ℓ	the training weeks that fall in leaf ℓ	
π_t, S	net return of week t ; Sharpe ratio per week	<code>net_returns</code> , <code>sharpe</code>
σ^2, ϱ	weekly variance of an asset; correlation between two assets	

Table 1: Notation. Weights are fractions of portfolio equity; returns are decimals ($0.01 = 1\%$).

1 What is the data?

1.1 What you supply

`load_data()` in `run.py` is a stub that you fill in. It returns four objects:

- **dates:** consecutive calendar days, with no gap (crypto trades every day). A gap raises an error.
- **closes:** a $(\text{days} \times n)$ array of positive daily closes, in the same asset order everywhere.
- **daily_features:** a $(\text{days} \times p)$ array. Row d must use information available at the close of day d only. NaN is allowed on days that are not decision dates (the warm-up), never on a decision date. The first decision falls 180 to 186 days after the first close, so a feature with a longer look-back raises an error.
- **current_weights:** your holdings now, for the live decision. They must be non-negative and sum to 1; **predict_weights** does not check it.

1.2 From daily data to weekly rows

Code: `weekly_rows`, `main.py:691`; `weekly_covariance`, `main.py:674`.

A day d becomes decision t when it falls on the chosen weekday (default Monday), has $H = 180$ daily returns behind it, and has a close seven days later. For each decision:

$$r_{t,i} = \frac{P_{d_t+7,i}}{P_{d_t,i}} - 1, \quad x_t = F_{d_t}, \quad \Sigma_t = 7 \widehat{\text{Cov}}(q_{d_t-H+1}, \dots, q_{d_t}) + \varepsilon I, \quad (1)$$

where $q_{d,i} = P_{d,i}/P_{d-1,i} - 1$ is the daily return, $\widehat{\text{Cov}}$ is the sample covariance (divisor $H - 1$), and $\varepsilon = 10^{-6}$ is a ridge. The factor 7 treats daily returns as independent and ignores compounding (a negligible understatement). `weekly_covariance` computes Σ for one day; the live decision uses the same function, so training and trading share one estimator.

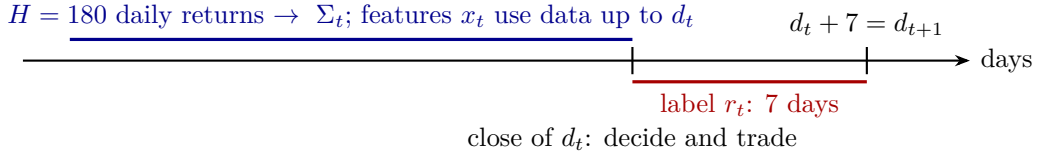


Figure 2: What one weekly row knows. Nothing to the right of d_t enters x_t or Σ_t .

Timing. Everything in x_t and Σ_t is known at the close of d_t ; the label r_t starts at that same close. The strategy is assumed to trade at that close. A unit test changes every price after a decision close and checks that the row's x_t and Σ_t do not move. *Your features are not checked:* their causality is your responsibility.

Sizes. Five years of daily data (1,826 days) give 234 weekly rows; ten years give 495. `train_and_test` holds out the last 52 rows, so training has 182 or 443 rows. The label of the last training row ends exactly at the first test decision, so nothing overlaps.

1.3 What is not in the data

- **No cash.** Portfolios are fully invested ($\mathbf{1}^\top w = 1$). A cash or stablecoin position can be added today as a constant-price column: the ridge keeps Σ_t positive definite, and the unchanged code runs (checked). Give that column a zero fee, `fee_rate=(f, f, 0)`: with a scalar `fee_rate`, a move from a coin to cash is charged on both legs, $2f$, twice the cost of the single trade it really is.
- **One asset is impossible:** the code needs $n \geq 2$, and with $n = 1$ the only feasible weight is 1.
- The weekly rows ignore the path inside the week, volumes (unless you put them in a feature), the risk-free rate, spreads and slippage.

2 Features: nature, frequency and choice

The code prescribes no feature. It only reads your array. The demo and the experiments use five: the 28-day momentum of assets 0 and 1, the 7-day momentum of asset 0, the 28-day volatility of asset 0, and a *pure-noise column used as a control*. The Wang–Hasuik articles feed a linear model with daily technical indicators per asset: log return, moving average (10 days in DIR), price bias, RSI (14 days in DIR), MACD difference, Bollinger-band width and a volume indicator. That list is not a recommendation for a tree: a moving average of prices is a level (see below).

2.1 What the pipeline does with a feature

- **Frequency.** Features are computed daily from daily data, but only their value at the weekly decision close is used: one row in seven. The label is the return over the *next seven days*. A look-back window of days to months is natural for that horizon.
- **Only the ordering matters.** A split is $x_j \leq \theta$, with θ halfway between two observed values. An increasing transformation of a feature (log, rank, scaling) gives the same partition of the training rows and the same leaf values, so no normalisation is needed. Only a new point that falls between the two training values around θ can be routed differently.
- **The model is piecewise constant.** A tree of depth 2 has at most four leaves, hence four predicted-return vectors. Features should therefore describe *states of the market* (regimes), not fine forecasts. Two nested splits capture an *interaction* of two features, for example “calm and rising” against “calm and falling”.
- **Use stationary quantities:** returns, ratios, volatilities. A threshold on a price level has no meaning out of sample.

2.2 Three constraints that should drive the choice

1. **Every feature adds candidate splits, and the best candidate looks good by luck.** With 182 training rows and a minimum leaf of 20, each feature brings 143 thresholds, so five features give 715 candidate splits at the root. On data with no signal, the best of them still removes 10% of the regret at $\lambda = 1$ (Table 7). Hence: few features (five or fewer), each with an economic reason. A pure-noise column is a cheap diagnostic, best used in a separate fit: in the demo it sits in the traded tree and wins the first split.
2. **With fully invested weights only differences matter.** Adding a constant to all predicted returns does not change the portfolio (Section 3.1). With BTC and ETH only, a feature is useful exactly to the extent that it predicts $r_{\text{ETH}} - r_{\text{BTC}}$. Relative features (ETH/BTC momentum, relative volatility) are the direct candidates. Market-level features (BTC trend, volatility regime) can also qualify, because the two coins do not have the same beta. What they cannot do without a cash column is reduce the exposure to the market.
3. **A feature that crosses its threshold often creates trading.** Each change of leaf can move the whole portfolio and pay fees on both legs. In one earlier two-asset run ($\lambda = 2$, fee 0.1%, earlier split rule), 73% (training) and 82% (test) of the fees came from leaf changes that did not correspond to a change of regime. Persistent features (longer windows, moving averages of returns) reduce this.

Candidates for BTC and ETH, to be validated on real data: momentum of each coin and of the ETH/BTC ratio over one to three months; realised-volatility regime; drawdown from the recent high; and, if the data is reliable, volume or funding-rate indicators. Real effects are likely to be interactions (a trend that works only in calm markets, for instance). That is the case a tree can represent and a one-feature rule cannot.

3 What the tree learns: the decision problem, the regret and the Sharpe ratio

3.1 The decision problem solved for every prediction

Code: PortfolioOptimizer.solve, main.py:176.

Given a predicted-return vector c , holdings u and a covariance Σ , the portfolio is

$$w^*(c; u, \Sigma) = \arg \max_{w \in \mathcal{W}} c^\top w - f^\top |w - u| - \lambda w^\top \Sigma w, \quad \mathcal{W} = \{w \in \mathbb{R}^n : 0 \leq w_i \leq \bar{w}, \mathbf{1}^\top w = 1\}. \quad (2)$$

With one fee for all assets and $\bar{w} = 1$ this is equation 8 of DIR, with the holdings u in the place of the previous portfolio; the code also allows one fee per asset and a cap. The optimizer is deterministic and is never trained.

Exact solution. At the optimum each weight is at 0, at $\bar{w}$, at u_i (fees put a kink there), or free strictly above or below u_i : 5^n patterns. Take a pattern with free set $\mathcal{F}$, fixed set $\mathcal{K}$ (its complement, with weights $w_{\mathcal{K}}$ fixed at 0, $\bar{w}$ or u_i) and sides $s_i = \text{sign}(w_i - u_i)$ for $i \in \mathcal{F}$. Stationarity under the budget constraint is a linear system:

$$A w_{\mathcal{F}} + \nu \mathbf{1} = g, \quad \mathbf{1}^\top w_{\mathcal{F}} = 1 - \mathbf{1}^\top w_{\mathcal{K}}, \quad A = 2\lambda \Sigma_{\mathcal{F}\mathcal{F}}, \quad g = c_{\mathcal{F}} - s_{\mathcal{F}} \circ f_{\mathcal{F}} - 2\lambda \Sigma_{\mathcal{F}\mathcal{K}} w_{\mathcal{K}}, \quad (3)$$

so $\nu = (\mathbf{1}^\top A^{-1} g - (1 - \mathbf{1}^\top w_{\mathcal{K}})) / (\mathbf{1}^\top A^{-1} \mathbf{1})$ and $w_{\mathcal{F}} = A^{-1}(g - \nu \mathbf{1})$. A pattern is admissible if all weights respect the bounds, every free weight lies on its assumed side of u_i , and, when no weight is free, the fixed weights sum to 1 (tolerance 10^{-9}). Every admissible candidate is a feasible portfolio, and the one with the highest objective is returned (then clipped to $[0, \bar{w}]$). Since $f \geq 0$, $\lambda > 0$ and $\Sigma \succ 0$, the objective is strictly concave: the optimum is unique, and its own pattern reproduces it, so the enumeration is exact. All rows are solved in one vectorised call. The 5^n cost is why $n \leq 4$.

Properties that matter later.

- *Shift invariance:* $w^*(c + \alpha \mathbf{1}) = w^*(c)$. Only differences between predicted returns matter.
- *Two assets a and b : closed form.* Let v be the holding of asset a , $\Delta c = c_a - c_b$, $\phi = f_a + f_b$, $\kappa = 2\lambda(\sigma_a^2 + \sigma_b^2 - 2\sigma_{ab})$ and $\bar{a} = (\sigma_b^2 - \sigma_{ab})/(\sigma_a^2 + \sigma_b^2 - 2\sigma_{ab})$ the minimum-variance weight. With $D = \Delta c - \kappa(v - \bar{a})$, the weight of asset a is

$$w_a = \begin{cases} v & \text{if } |D| \leq \phi \quad (\text{no trade}), \\ \bar{a} + \frac{\Delta c - \phi \operatorname{sign}(D)}{\kappa} & \text{otherwise,} \end{cases} \quad \text{then clipped to } [1 - \bar{w}, \bar{w}]. \quad (4)$$

(Checked against the solver on 2,000 random cases, to 10^{-14} .) So a leaf acts through *one* number, Δc ; the response is linear with slope $1/\kappa$; and the no-trade band has half-width ϕ (0.2% with the default fees).

- *Scale.* With equal variances σ^2 , correlation ρ and no fees, (4) gives $w_a = \frac{1}{2} + \Delta c / (4\lambda\sigma^2(1 - \rho))$: the portfolio is all-in as soon as $\Delta c \geq 2\lambda\sigma^2(1 - \rho)$. With 8% weekly volatility this gap is $0.51\% \times \lambda$ for $\rho = 0.6$ and $0.26\% \times \lambda$ for $\rho = 0.8$ (closer to BTC and ETH). At $\lambda = 1$ almost any prediction gives an all-in portfolio.

3.2 The loss: decision regret

Code: `utility, main.py:168; _score_prediction, main.py:560.`

The realised utility of a decision w in week t , and the regret (SPO loss) of a prediction c , are

$$U_t(w; r_t) = r_t^\top w - f^\top |w - u_t| - \lambda w^\top \Sigma_t w, \quad (5)$$

$$\operatorname{Reg}_t(c) = U_t(w^*(r_t; u_t, \Sigma_t); r_t) - U_t(w^*(c; u_t, \Sigma_t); r_t) \geq 0. \quad (6)$$

The first term is the best decision in hindsight, *from the same holdings* u_t . Fees and the covariance penalty are inside the regret (DIR eq. 10): a prediction is judged on the objective the optimizer really maximises. The benchmark is a one-week oracle, as in the articles; it does not plan over several weeks. Numerically, a row regret below -10^{-6} (`regret_tolerance`) aborts the fit; negative values between -10^{-6} and 0 are set to zero.

3.3 The path and the Sharpe ratio

Code: `replay_path, main.py:301; _trade, main.py:350.`

A sequence of predictions $c_1, \dots, c_T$ is traded in time order, starting from $u_1 = \mathbf{1}/n$ (or from given holdings):

$$w_t = w^*(c_t; u_t, \Sigma_t), \quad \pi_t = r_t^\top w_t - f^\top |w_t - u_t|, \quad u_{t+1} = \frac{w_t \circ (1 + r_t)}{1 + r_t^\top w_t}. \quad (7)$$

Holdings *drift* with returns between two decisions, so keeping a constant target also costs fees. The Sharpe ratio is $S = \operatorname{mean}(\pi) / \operatorname{std}(\pi)$ per week (divisor $T - 1$, no risk-free rate). The replay also returns the regret (6) of every week along the path.

3.4 What the tree is supposed to learn

The tree is a piecewise-constant map $x \mapsto \hat{c}_{\ell(x)}$. The leaf vector $\hat{c}_\ell$ is *not* trained to forecast returns accurately. It is trained so that the portfolios $w^*(\hat{c}_\ell; u_t, \Sigma_t)$ have a small regret on the weeks of the leaf. DIR calls such values “decision-inducing scores”: they need not be calibrated forecasts. Here they are searched inside $[-b, b]^n$; DIR instead clips its predictions to ± 0.1 after training.

Link with SPOT. SPOT’s Theorem 1 says that the leaf mean of the realised vectors minimises the leaf’s SPO loss when the optimal decision for that mean is unique (always true here). It extends to objectives $c^\top w - h(w)$ *when all rows of a leaf share the same h* , that is, the same u_t and Σ_t . Here they differ by row, so the mean is only a starting point. A numerical check agrees: no random candidate beats the mean when u and Σ are shared, and many do when either varies by row.

4 How the tree is built and trained

4.1 Fitting one leaf

Code: `_fit_leaf`, `main.py:569`.

For a leaf with rows I_ℓ , with the holdings u_t *frozen* at the last replay of the current tree:

$$\hat{c}_\ell \approx \arg \min_{c \in [-b, b]^n} L_\ell(c), \quad L_\ell(c) = \sum_{t \in I_\ell} \text{Reg}_t(c). \quad (8)$$

The search is deterministic:

1. Start from the leaf's mean return, clipped to $[-b, b]$. If a *fallback vector* is given, evaluate it too and take it only if it is strictly better. The fallback is the refitted parent for a child, the leaf's own stored vector for a refit, and nothing for the root.
2. Run 3 passes (`search_passes`). In pass k , for each asset in turn, evaluate $G = 7$ equally spaced values of that coordinate (`search_grid_size`) and move to the best one if it is strictly better. Pass 0 spans $[-b, b]$; pass $k \geq 1$ spans the current best $\pm 2b/(G-1)^k$, inside $[-b, b]$.

With $b = 0.10$ the windows are ± 0.1 , ± 0.033 and ± 0.0056 , and the final step is about 0.002. One fit costs $2 + 3nG$ evaluations of L_ℓ (65 for $n = 3$), each one vectorised solve over the leaf. Against a dense search, the result is within 0.03% (median) of the best regret found, 2.1% at worst; the plain mean is 0.8% above (median).

4.2 Growing the tree

Code: `fit`, `_best_split`, `_shortlist` and `_refit_leaves` (`main.py`, lines 411, 488, 525 and 545).

Growth is *best-first*: one split per round, chosen among all leaves, because the holdings couple the leaves.

Algorithm.

0. **Root.** Fit one vector on all rows as if every week started from equal weights. Replay the tree: this gives the holdings u_t it would really have had, the hindsight utilities, and the Sharpe ratio S .
1. **For each leaf** with depth $< \text{max_depth}$ and at least $2m$ rows ($m = \text{min_samples_leaf}$):
 - a. *Parent refit.* Refit the leaf itself on the current holdings: $\hat{c}_\ell^{\text{refit}}$. It is only a yardstick and the children's fallback; it is never written into the tree.
 - b. *Screen.* For every feature j and threshold θ , give each child its clipped mean return and compute $Q = L_{\text{left}}(\bar{r}_{\text{left}}) + L_{\text{right}}(\bar{r}_{\text{right}})$. Keep the `shortlist_size` = 3 smallest Q .
 - c. *Regret reduction.* For each shortlisted split, fit both children with (8) and compute $\Delta = L_\ell(\hat{c}_\ell^{\text{refit}}) - L_{\text{left}}(\hat{c}_{\text{left}}) - L_{\text{right}}(\hat{c}_{\text{right}}) \geq 0$. Keep the split if $\Delta/|I_\ell| > \text{min_regret_improvement}$ (10^{-6}).
2. **Choice and confirmation.** Sort the kept splits of all leaves by Δ , largest first (SPOT's criterion). Replay the whole tree with the first one. If its Sharpe ratio $S' > S + \text{min_sharpe_improvement}$, accept it; otherwise try the next. If none passes, stop.
3. After an acceptance, the replay of the accepted tree gives the new u_t , hindsight utilities and S . Return to step 1.
4. **Final refit.** Refit all leaves on the final holdings; keep the refit only if the replayed Sharpe ratio does not fall.

In one formula, with $\mathcal{C}$ the shortlisted splits (ℓ, j, θ) of all leaves and $\mathcal{T} \oplus (\ell, j, \theta)$ the tree $\mathcal{T}$ with that split added:

$$(\ell, j, \theta)^* = \arg \max_{(\ell, j, \theta) \in \mathcal{C}} \Delta_\ell(j, \theta) \quad \text{subject to} \quad \frac{\Delta_\ell(j, \theta)}{|I_\ell|} > \varepsilon_R \quad \text{and} \quad S(\mathcal{T} \oplus (\ell, j, \theta)) > S(\mathcal{T}) + \varepsilon_S, \quad (9)$$

where ε_R and ε_S are `min_regret_improvement` and `min_sharpe_improvement`, and $S(\cdot)$ is the weekly Sharpe ratio of the *whole* replayed training path. The total Δ ranks; Δ per row gates; the Sharpe ratio confirms.

Details.

- *Thresholds* (`_thresholds`, `main.py:597`): midpoints between consecutive distinct values, kept when both children have at least m rows. If `max_thresholds` is set, an evenly spaced subset by rank is used.
- *Why the parent is refitted*: a leaf's vector was fitted on the holdings of an earlier tree, while its children are fitted on the current ones. Without the refit, part of Δ would only be the benefit of re-tuning. The baseline S is still the Sharpe ratio of the tree as it is stored.
- *Frozen holdings*. Within a round, regrets are measured on the holdings of the tree *without* the candidate split. The change of path that a split causes is seen by the Sharpe replay, and by the next round.
- *What a final tree contains*. A leaf keeps the vector of the round that created it, unless the final refit is kept (at $\lambda = 1$ it changed a vector in 38 of 40 fits and was then kept in 23, with no systematic effect on the regret or the Sharpe ratio).
- *Both tests are in-sample*. Δ and S' are measured on the training weeks (Section 8).
- *There is no single global objective*: leaf values minimise regret; the structure follows the largest regret reduction; the Sharpe ratio confirms. The search is greedy.
- *Cost* on one core, 3 assets, 5 features, depth 2: about 6s for 182 rows. For ten years (443 rows, 2,020 root candidates) about 25s with the settings of `run.py`, and 11 to 16s with `max_thresholds=100`.

4.3 Parameters

Parameter	Default	<code>run.py</code>	Role
<code>max_weight</code> $\bar{w}$	1.0	1.0	cap per asset; long-only, fully invested
<code>fee_rate</code> f	0.001	0.001	fee per traded notional, scalar or one per asset; same value in optimizer, regret and replay
<code>risk_aversion</code> λ	1.0	1.0	multiplies $w^\top \Sigma w$; its scale decides whether the covariance matters
<code>max_depth</code>	4	2	root is depth 0
<code>min_samples_leaf</code> m	10	20	minimum weeks per leaf
<code>max_thresholds</code>	10	None	candidate thresholds per feature; None tests all
<code>shortlist_size</code>	3	3	screened splits per leaf that get a full fit
<code>min_regret_improvement</code> ε_R	10^{-6}	10^{-6}	gate on $\Delta/ I_\ell $; rejects almost nothing
<code>min_sharpe_improvement</code> ε_S	0	0	margin on the weekly training Sharpe ratio
<code>prediction_bound</code> b	0.10	0.10	leaf vectors in $[-b, b]^n$
<code>search_passes</code> , <code>search_grid_size</code>	3, 7	3, 7	leaf search
<code>regret_tolerance</code> , <code>verbose</code>	10^{-6} , off	10^{-6} , on	numerical guard on row regrets; printing of the growth log
<code>weekday</code> , <code>window</code> , <code>ridge</code>	0, 180, 10^{-6}	same	decision day, covariance window H , ridge ε
<code>test_periods</code>	52	52	weeks held out at the end

Table 2: Parameters. The class defaults of depth and leaf size are looser than `run.py`; the default of 10 thresholds per feature is tighter than `run.py`, which tests all of them.

5 What comes out of the tree, and how it is used

The fitted object is a binary tree. Internal nodes hold a feature index and a threshold; leaves hold $\hat{c}_\ell$. `growth_log` lists the accepted splits: depth, size of the split leaf, feature, threshold, regret

reduction per row *of that leaf* (against the refitted leaf), and the weekly training Sharpe ratio before and after. The training arrays are deleted after fitting. The demo (`python main.py`) prints:

```
root: training Sharpe 0.1675
split 1: depth=0, N=182, x[4] <= 0.742, regret -0.00527 per row, Sharpe 0.1675 -> 0.2147
split 2: depth=1, N=133, x[0] <= 0.1336, regret -0.00222 per row, Sharpe 0.2147 -> 0.2357
split 3: depth=1, N=49, x[2] <= -0.02425, regret -0.00163 per row, Sharpe 0.2357 -> 0.2475
refit leaf scores: Sharpe 0.2475 -> 0.2406 (discarded)
if noise <= 0.742002:
    if momentum_28d_a0 <= 0.133599:
        leaf: N=103, scores=[ 0.003596  0.030422 -0.002701]
    else:
        leaf: N=30, scores=[ 0.008959 -0.00263  0.001669]
else:
    if momentum_7d_a0 <= -0.02425:
        leaf: N=21, scores=[ 0.011834 -0.016961  0.007677]
    else:
        leaf: N=28, scores=[-0.016145 -0.00978  0.      ]
```

Note that the first split is on the *noise* feature (`x[4]`): an example of the in-sample luck of Section 8. **Three calls use the tree.**

- `predict_returns(X)` routes each row ($x_j \leq \theta$ goes left) and returns $\hat{c}_{\ell(x)}$.
- `predict_weights(X, U, Sigma)` returns $w^*(\hat{c}_{\ell(x)}; u, \Sigma)$ from (2). Holdings and covariance are supplied separately from the features.
- `replay(X, R, Sigma, initial_weights)` trades the tree through a period with (7).

The weekly live procedure (end of `run.py`), at the close of the decision weekday:

1. compute today's feature row x from data up to the close;
2. compute $\Sigma = \text{weekly_covariance}(\text{closes}, \text{last day})$;
3. read the account's current weights u (they have drifted since last week);
4. trade to $w^*(\hat{c}_{\ell(x)}; u, \Sigma)$.

Which tree trades. In `run.py` the live decision uses the tree returned by `train_and_test`, fitted on all rows *except the last 52*: it has never seen the most recent year. Nothing is saved, so every run refits, and `run.py` does not check that the last day is the decision weekday. To trade a tree trained on all the history, fit one more tree on all rows after the backtest (W10 in Section 8).

How to read the leaf values. They are inputs to the optimizer, not forecasts. Only their differences matter, and their scale must be read against $2\lambda\sigma^2(1 - \varrho)$ (Section 3.1). In the first leaf above, the gap between assets 1 and 0 is 2.7%, far above the 0.5% that makes the portfolio all-in at $\lambda = 1$: all 103 weeks of that leaf are all-in on asset 1. Over the whole demo tree, 79% of the training portfolios are all-in (largest weight above 0.99). The leaves with $N = 30$ and $N = 21$, whose gaps between assets 0 and 2 are only 0.7% and 0.4%, hold a mix of these two assets; the $N = 28$ leaf (gap 1.0%) is all-in on asset 2. At $\lambda = 1$ the tree is close to “pick one asset per leaf”.

6 How to backtest a tree and the strategy

6.1 The replay is the backtest

`replay_path` applies (7) week by week and returns a `PathResult` with, for every week: holdings u_t , weights w_t , turnover $\|w_t - u_t\|_1$, fees $f^\top |w_t - u_t|$, gross return $r_t^\top w_t$, net return π_t , utility (5)

and regret (6); plus the final holdings. `summary()` annualises with 52 periods per year:

$$\begin{aligned} \text{return} &= \left(\prod_{t=1}^T (1 + \pi_t) \right)^{52/T} - 1, & \text{volatility} &= \sqrt{52} \text{std}(\pi), \\ \text{Sharpe} &= \sqrt{52} \frac{\text{mean}(\pi)}{\text{std}(\pi)}, & \text{fees per year} &= 52 \text{mean}_t (f^\top |w_t - u_t|), \\ \text{max drawdown} &= \min_t \left(\frac{V_t}{\max_{0 \leq s \leq t} V_s} - 1 \right), & V_t &= \prod_{s \leq t} (1 + \pi_s), \quad V_0 = 1, \end{aligned}$$

and turnover is the mean of $\|w_t - u_t\|_1$ per week (a full swap between two assets counts 2).

6.2 The protocol of `run.py`

Code: `train_and_test, main.py:734`; `performance_report, main.py:764`.

1. Build the weekly rows. Train on all rows except the last 52.
2. Fit the tree, and the *same tree without splits* (`max_depth=0`).
3. Replay three strategies on the training rows from equal weights: the tree, the tree without splits, and an equal-weight portfolio rebalanced every week (`replay_constant_weights`, same drift and fee accounting).
4. Replay the 52 test rows, each strategy *continuing from its own final training holdings*.
5. `performance_report` prints both tables.

The tree without splits isolates what the splits add. Table 3 shows the demo.

	Train, 182 weeks						Test, 52 weeks					
	ret.	vol.	Sharpe	max DD	turn.	fees	ret.	vol.	Sharpe	max DD	turn.	fees
SPO tree	141%	59%	1.78	−33%	1.06	5.5%	44%	64%	0.88	−33%	0.76	4.0%
Tree without splits	70%	57%	1.21	−50%	0.01	0.0%	10%	61%	0.45	−24%	0.00	0.0%
Equal weight	13%	47%	0.48	−44%	0.04	0.2%	31%	54%	0.76	−26%	0.04	0.2%

Table 3: Output of `python main.py`: one synthetic dataset, $\lambda = 1$, current rule. Annualised figures; turnover per week; fees per year. One dataset proves nothing: over 20 datasets of the same kind the tree’s test Sharpe ratio is 0.14 (s.e. 0.11) below that of equal weight (Table 5).

6.3 Conventions and limits of this backtest

- The strategy trades at the daily close it decides on. Fees are a fraction of pre-trade equity, subtracted from the week’s return; the fee $\times$ return cross term (about 5×10^{-5} per week) is ignored. There is no spread, slippage or funding, and no risk-free rate in the Sharpe ratio.
- There is a single train/test split. The Wang–Hasuik articles retrain every month on a rolling window with a validation period; the code does not (Section 8).
- **A Sharpe ratio measured on one year is very imprecise.** For a weekly Sharpe ratio S estimated on T weeks the standard error is $\sqrt{(1 + S^2/2)/T}$, hence about $\sqrt{52/T}$ once annualised: 1.0 for 52 weeks, 0.58 for three years, 0.45 for five. A difference of 0.2 between two strategies cannot be seen on one test year.

7 Tests done, results, and what to expect

7.1 Is the code correct?

Test file	Tests	What it proves
<code>test_optimizer.py</code>	7	the solver is never beaten by a dense grid (2, 3 assets) nor by SLSQP (2–4 assets), also when holdings drifted above the cap; two-asset closed form; no-trade zone; row independence; input validation
<code>test_replay.py</code>	7	a drift and fee example computed by hand; every accounting identity on a random path; a path continued from its final holdings
<code>test_tree.py</code>	10	splits on a clean regime feature; rejects weekly flipping once $2 \times \text{fee}$ exceeds the edge; the largest regret reduction wins among Sharpe-raising splits; children are compared with the refitted parent; both gates; a worse final refit is discarded; depth and leaf limits
<code>test_data.py</code>	7	weekday and spacing; r_t and Σ_t against hand computations; <i>no look-ahead</i> when later prices change
<code>test_report.py</code>	4	drawdown, annualisation, equal-weight baseline, holdings carried from train to test

Table 4: The 35 unit tests (`python -m unittest discover -s tests`, about 5 s).

- **Planted bugs.** 20 deliberate mutations each make at least one test fail. *Replay*: no drift; fees left out of net returns; wrong divisor in the Sharpe ratio. *Tree*: Sharpe check removed; regret gate removed; depth limit ignored; a worse final refit kept; highest Sharpe chooses instead of largest Δ ; children start from the leaf’s stored vector instead of the refitted one; Δ measured against the stored vector instead of the refitted one. *Data*: covariance window including the next day; features read a day late; label starting a day late; wrong weekday arithmetic; covariance not scaled to a week. *Report*: drawdown ignoring the starting wealth; uncompounded return; fees summed instead of per year; equal weight never rebalanced; test period restarted from equal weights.
- **Audit checks beyond the unit tests** (run with the earlier split rule, see below): the final tree routes every training row to the leaf it had during growth; the logged Sharpe ratio equals the replayed one; the regret minimised in training equals the regret of the replayed path to 9 decimals; fitting twice gives the same tree; adding a constant to all predictions moves no weight by more than 3×10^{-15} . The solver agrees with SLSQP to 5×10^{-14} .

7.2 Experiments: design

All experiments use **synthetic** daily data for three assets with correlations 0.5 to 0.6, 5 or 10 years, the five features of Section 2, 20 datasets per cell unless stated, and the last 52 weeks as test. The settings are those of `run.py`, except λ (given in each table) and `max_thresholds=100` on the ten-year data, to limit run time. Three generators:

- *Single-feature signal* (3% daily volatility): a hidden regime, redrawn every 60 days, tilts the drifts of assets 0 and 1 in opposite directions (+0.2% against -0.1% a day, an expected gap of 2.1% per week). The 28-day momentum reveals it partly. The three momentum features count as *informative*.
- *Interaction signal*: the best asset depends on a volatility regime (90-day blocks) *and* a trend (60-day blocks). In calm periods the trend gives a drift of +0.2% a day to asset 0 (up-trend) or to asset 1 (down-trend). Assets 0 and 1 have 2% daily volatility in calm periods and 4.5% in stressed ones, where they lose 0.2% a day. Asset 2 is defensive: 0.4 times their volatility, and +0.05% a day in stress. Every feature except the noise column counts as informative.
- *No signal* (3% daily volatility): returns are independent of every feature.

“Momentum rule” feeds the optimizer with +0.1 for whichever of assets 0 and 1 has the higher 28-day momentum, −0.1 for the other and 0 for asset 2. It is all-in at $\lambda = 1$; at $\lambda = 10$ it is all-in on the single-feature data but only in about a third of the weeks on the interaction data; at $\lambda = 50$ it is a tilt.

Two versions of the split rule. Until 17 September 2026 the code chose, among the shortlisted splits, the one with the *highest replayed Sharpe ratio*, and measured Δ against the leaf’s old vector. The current rule is that of Section 4. *Current rule:* the demo, Tables 5, 6 and 8, the positive control and the final-refit count. *Earlier rule:* the audit checks, the acceptance shares, the pure-noise margin test and the fee break-even. Table 7 comes from a separate probe of the root that already measures the Δ of the current rule. Table 5 and the positive control were run with both rules, with the same conclusions.

7.3 Results

Setting	λ	Tree minus ...			Tree, median train / test
		equal weight	tree without splits	momentum rule	
5 years, single-feature signal	1	−0.14 (0.11)	+0.08 (0.14)	−0.38 (0.19)	1.34 / 0.26
	10	+0.06 (0.04)	+0.06 (0.05)	−0.19 (0.13)	0.88 / 0.61
	50	+0.03 (0.02)	+0.01 (0.01)	−0.20 (0.08)	0.34 / 0.64
5 years, interaction signal	1	+0.33 (0.21)	+0.10 (0.19)	+0.52 (0.25)	1.42 / 0.45
	10	+0.46 (0.18)	+0.08 (0.07)	+0.52 (0.20)	0.62 / 0.76
	50	+0.42 (0.20)	+0.01 (0.01)	0.00 (0.10)	0.32 / 0.72
10 years, interaction signal	1	+0.81 (0.16)	+0.12 (0.14)	+0.84 (0.22)	1.10 / 0.37
	10	+0.66 (0.21)	−0.03 (0.03)	+0.51 (0.21)	0.43 / −0.04
	50	+0.69 (0.21)	−0.01 (0.01)	+0.08 (0.06)	0.26 / 0.18

Table 5: Current rule, 20 datasets per row. Columns 3 to 5: mean of the paired differences of annualised test Sharpe ratios, standard error in brackets. Last column: median annualised Sharpe ratio of the tree on the training weeks and on the test year.

- **What the splits add is small and not distinguishable from luck.** The column “tree without splits” is positive in 7 of 9 rows but never beyond 1.3 standard errors, and Table 6 shows +0.11 (0.06), that is 1.8 standard errors, with no signal at all. Where the tree beats equal weight (interaction data), the tree *without splits* beats it almost as much: a single leaf already learns to hold the defensive asset.
- **Where a simple signal exists, a two-line rule does better.** On the single-feature data the momentum rule is ahead of the tree at every λ , by 0.19 to 0.38 (1.4 to 2.5 standard errors). The tree is ahead of that rule only on the interaction data, and there the tree without splits is ahead of it by almost as much: the rule never favours the defensive asset.
- **On the single-feature data, raising λ removes the damage of overfitting, but it does not make the splits useful.** From $\lambda = 1$ to 10 the median train/test Sharpe ratios go from 1.34/0.26 to 0.88/0.61, test-year fees from 3.5% to 1.9% a year, and all-in test weeks from 76% to 0%. At $\lambda = 50$ the tree and the tree without splits have the same test Sharpe ratio to within 0.01. On the ten-year interaction data, $\lambda = 1$ gave the best test figures.
- With the earlier rule the same table gave the same reading (first row: −0.10, +0.12, −0.35), as did two settings measured only with the earlier rule at $\lambda = 1$: no signal, 5 years, +0.06 (0.13) against equal weight; single-feature signal, 10 years, +0.28 (0.14) against equal weight and −0.20 (0.14) against the momentum rule.

Data ($\lambda = 10$)	Train: flat $\rightarrow$ tree		Test: flat $\rightarrow$ tree		Test Sharpe, tree – flat
	regret	Sharpe	regret	Sharpe	
No signal, 5 years	–8%	0.03 $\rightarrow$ 0.55	+6%	0.40 $\rightarrow$ 0.45	+0.11 (0.06)
Signal, 5 years	–10%	0.31 $\rightarrow$ 0.88	+7%	0.68 $\rightarrow$ 0.61	+0.06 (0.05)
Signal, 10 years	–3%	0.14 $\rightarrow$ 0.47	+1%	0.70 $\rightarrow$ 0.73	+0.03 (0.03)
Signal, 5 years, half the noise	–16%	0.95 $\rightarrow$ 1.73	+8%	0.59 $\rightarrow$ 0.45	+0.01 (0.12)
Signal, 10 years, half the noise	–12%	0.65 $\rightarrow$ 1.43	–4%	1.45 $\rightarrow$ 1.53	+0.30 (0.12)

Table 6: When do leaves help? Current rule, $\lambda = 10$, single-feature generator, 20 datasets per row; “flat” is the tree without splits; “half the noise” is 1.5% daily volatility. Columns 2 to 5 compare *medians over the 20 datasets*, flat $\rightarrow$ tree: relative change of the median (over datasets) of the mean weekly regret, and the two median annualised Sharpe ratios. The last column is the *mean of the paired differences* with its standard error; it need not have the sign of the difference of medians. In training, leaves always help, even with no signal. On the test year they make the regret worse, except in the last row, where the first split is on a momentum feature in 20 of 20 datasets. Note the +0.11 (0.06) obtained with no signal at all: it is the yardstick for reading the other rows.

Gain of the best root split, $\Delta/L_\ell(\hat{c}_\ell^{\text{refit}})$ (medians over 16 datasets, 5 years)	$\lambda = 1$	$\lambda = 10$
No signal: the best split, by luck (90th percentile in brackets)	10.2% (14.0%)	3.4% (5.3%)
Single-feature signal, best split on an informative feature	8.5%	3.0%
Interaction signal, best split on an informative feature	9.0%	3.4%
A sixth feature that reveals the true regime exactly	14.3%	5.7%

Table 7: Regret reduction by luck and by signal, from a probe of the root that fully fits the 12 best-screened splits plus the best-screened split of every other feature. Realistic signals do not stand out from luck with five years of data. Take as threshold the 90th percentile of luck (2 of the 16 no-signal datasets exceed it by construction) and require the Sharpe ratio to rise. At $\lambda = 1$ a split is then accepted in 2 of 16 single-feature datasets, 2 of 16 interaction datasets and 9 of 16 datasets with the exact regime feature; 12 of these 13 splits are on an informative feature. So about 85% of the realistic signals are missed, and half of the exact ones.

λ	Weekly averages on the training path				All-in portfolios		
	gross return	fees	risk penalty $\lambda w^\top \Sigma w$	regret	hindsight	tree	turnover
1	0.064	0.0008	0.006	0.034	91%	68%	0.77
10	0.056	0.0003	0.045	0.027	31%	0%	0.32
50	0.054	0.0001	0.212	0.009	0%	0%	0.11

Table 8: Size of each term of the utility (5): current rule, single-feature data, means over 5 datasets. At $\lambda = 1$ the covariance penalty is a tenth of the returns; at $\lambda = 50$ it is four times larger.

Other measurements.

- *In-sample acceptance is not a filter* (earlier rule, $\lambda = 1$). At the root of 10 datasets, between 32% and 99.7% of the 715 candidates (median 82%) both reduce the regret and raise the training Sharpe ratio. In a separate set of 20 no-signal datasets, 20 of 20 trees still split (also with the current rule at $\lambda = 10$, Table 6).
- *Size of the Sharpe improvement obtained by luck*. On no-signal data the best root split raises the *weekly* training Sharpe ratio by a median of 0.061 at $\lambda = 1$ and 0.031 at $\lambda = 10$ (0.44 and 0.22 annualised), and the gain is positive in 16 of 16 datasets. In a pure-noise test (earlier rule, 20 datasets, depth 1) a margin ε_S of 0.02 blocked no split, 0.05 blocked 3 of 20, and 0.10 blocked 17 of 20. But a margin that blocks luck also blocks real splits: in the demo the informative split added only 0.021. A fixed margin is therefore no more a usable filter than the regret threshold.
- *Positive control* (current rule, $\lambda = 1$, 20 datasets). A sixth feature reveals the hidden regime exactly. At 3% daily volatility the tree splits on it first in 13 of 20 datasets; in 6 of the 7

misses the split was not in the shortlist of 3. At 1.5% it is first in 18 of 20. There, at depth 1, the tree’s annualised test Sharpe ratio is 0.06 (s.e. 0.06) *below* the rule that knows the regime (it feeds the optimizer with +0.1 for the asset the true regime favours and −0.1 for the two others) and 1.60 above equal weight; at depth 2, the setting of `run.py`, it is 0.24 (s.e. 0.07) below that rule, and at 3% volatility 0.57 (s.e. 0.15) below. The machinery works when the signal is identifiable, but further splits still cost.

- *Fees and flipping.* A split that swaps two assets every week is kept only when the weekly edge exceeds $2 \times \text{fee}$ (break-even measured at a fee of 0.4% for an edge of 0.8%).

7.4 What you can expect

- **From the code:** it is a faithful, tested implementation of the design. Results are reproducible and the backtest accounting is exact under its conventions.
- **From the strategy, as it stands:** no evidence that the splits add an edge. With about 180 to 440 noisy weekly rows and hundreds of candidate splits, the tree fits luck as easily as signal. Training figures are strongly optimistic: at $\lambda = 1$ the median Sharpe ratio is 1.1 to 1.4 in training against 0.3 to 0.5 on the test year (Table 5). On a simple signal the tree loses to a two-line momentum rule; on an interaction signal it beats simple rules, but mostly through what a single leaf learns.
- **From real data:** unknown. Five to ten years of weekly data can only confirm a large edge, because of the precision of the Sharpe ratio (Section 6). A useful tree needs few, persistent, well-motivated features, a meaningful λ , and an acceptance test on weeks it did not learn from.

8 Known weaknesses and ideas to resolve them

Table 9: Weaknesses found so far, with the evidence, and the ideas to resolve them. None of the ideas is implemented, except that `max_thresholds` already exists.

	Weakness (evidence)	Ideas
W1	Splits are accepted on the training weeks. The best of ~ 700 candidates wins by luck: it removes 10% of the regret with no signal ($\lambda = 1$), and 20 of 20 no-signal trees split. At $\lambda = 1$ the training Sharpe ratios (1.1 to 1.4) are far above the test ones (0.3 to 0.5, Table 5).	Accept a split only if the regret also improves on later weeks the tree did not see (SPOT prunes on a 20% validation set; the Wang–Hasuike articles keep a validation window). On one short window this test is weak too: between two versions of a tree, the one-year Sharpe difference varies across datasets by 0.1 to 0.8 for an effect near 0.1 (Table 5), so the regret must carry it. Fewer features and larger leaves; thinning the thresholds alone helps little, because neighbouring thresholds give almost the same partition.
W2	The regret threshold (10^{-6}) rejects nothing, and its useful value depends on λ , on T and on the number of candidates (14% at $\lambda = 1$, 5% at $\lambda = 10$). A fixed Sharpe margin fails in the same way.	Express the threshold as a share of the refitted parent’s regret and calibrate it on the data: permute the feature rows against the returns in blocks (to keep the persistence of the features), record the best lucky gain, use its 90th percentile. Not tested yet. Expect few false alarms and a low recall (Table 7).

	Weakness (evidence)	Ideas
W3	$\lambda = 1$ switches the covariance off. The weekly risk penalty is a tenth of the returns and about 70 to 80% of the portfolios are all-in (68% in Table 8, 79% in the demo); turnover 0.5 to 1.1 per week; fees 2.4 to 5.5% a year.	Choose λ on validation data, together with step (1) below. $\lambda \approx 10$ did better than 1 in two of the three settings of Table 5 (not on the ten-year interaction data); the rule $2\lambda\sigma^2(1 - \varrho)$ suggests 10 to 30 for fully invested BTC and ETH (not tested). <i>With a cash column the scale changes:</i> the gap that matters is coin against cash, $2\lambda\sigma^2 \approx 1.3\% \times \lambda$, five times larger, so λ must be about five times smaller. At $\lambda = 50$ the splits no longer matter. (DIR finds that raising λ does not cut turnover, even with its predictions clipped to ± 0.1 . Here it does, Table 8, and the bound b is not the reason: with $b = 1$ the turnover falls in the same way, 0.78, 0.35, 0.11.)
W4	Noise switches. A split that is right on average still changes leaf when its feature wobbles around the threshold (the TODO in <code>main.py</code> ; fee shares in Section 2).	Partial adjustment $w_t = u_t + \delta(w^* - u_t)$ (DIR eq. 35, its most effective turnover control); a buffer zone around thresholds; smoother features; a turnover penalty in the path score.
W5	Always fully invested. One coin is impossible; two coins can only rotate, so the drawdown cannot fall below that of the assets held. On the test year the tree's median max drawdown was -34% to -41% on the single-feature data (equal weight -36%). It was smaller on the interaction data (-21% to -36% , equal weight -39%) only because one synthetic asset is defensive; BTC and ETH offer no such asset.	Add a constant-price cash or stablecoin column with a zero fee, <code>fee_rate=(f, f, 0)</code> (works with the unchanged code). λ then also sets the exposure: at $\lambda = 10$ a test portfolio held 87% cash.
W6	One train/test split, one noisy year. Standard error of a one-year Sharpe ratio: 1.0.	Walk-forward: retrain monthly or quarterly on 10 years, stitch the out-of-sample weeks, compare with buy-and-hold, 50/50, a momentum rule and a placebo with shuffled features; bootstrap intervals.
W7	Few rows: 182 to 443 weekly rows, and they depend on the chosen weekday.	Your idea: one row per day with the next-7-day return, replayed as 7 interleaved weekly paths (the W–H SPO article also appears to train on daily samples while rebalancing monthly; it does not state the label horizon). Rows overlap by 6/7, so the gain is far below $7\times$: close to $1\times$ for persistent features (about $1.1\times$ for a 28-day feature and $1.4\times$ for a 7-day feature in a quick simulation). It removes the weekday dependence and gives 7 test paths. Leaf sizes must then count days, and training and test need a 7-day gap. Not tested on the tree.
W8	Approximations in training. Holdings frozen within a round; root fitted from equal weights; the final refit is neutral on average; the shortlist of 3 can exclude the right split (the exact-regime split, in 6 of 20 datasets); leaf search up to 2.1% above the best.	Acceptable as designed. Enlarge the shortlist only together with W1, since more candidates mean more luck.

	Weakness (evidence)	Ideas
W9	Backtest and estimation conventions. Trades at the decision close; no spread or slippage; no risk-free rate; Σ_t is a plain 180-day sample covariance scaled by 7; class defaults of depth 4 and leaf 10 looser than <code>run.py</code> ; at most 4 assets (5^n patterns).	Add an execution delay and a spread; align the defaults of depth and leaf size with <code>run.py</code> ; report the Sortino ratio as the W–H SPO article does; try an exponentially weighted covariance.
W10	The live tree is one year old. <code>run.py</code> trades the tree fitted without the last 52 weeks, refits at every run and saves nothing; live holdings and the weekday are not validated.	After the evaluation, refit on all rows and save the tree; retrain monthly rather than at each run (the walk-forward of W6 does this); validate the live inputs.

Suggested order. (1) Real BTC and ETH data with a cash column, walk-forward, against simple baselines and a placebo, with λ chosen on validation data (W3, W5, W6): it shows whether your features carry a signal large enough to be seen (a null result there does not prove that there is none). (2) Out-of-sample acceptance of splits (W1) and a calibrated threshold (W2). (3) Turnover control (W4). (4) Refit on all rows for live trading (W10). (5) Daily-sampled rows (W7).

A Relation to the articles

Element	Article	Code
Decision problem, regret	DIR eq. 8 and eq. 10 on the simplex	identical with a scalar fee and $\bar{w} = 1$; holdings drift instead of staying at the previous target
Risk term	W–H SPO article: $\lambda \ w\ _2^2$; DIR: $\lambda w^\top \Sigma w$	$\lambda w^\top \Sigma w$
Covariance	DIR: 220 trading days, regularisation 10^{-6}	180 calendar days $\times 7$, ridge 10^{-6}
Training loss	both Wang–Hasuike articles: SPO+ surrogate, linear model, gradients	true regret minimised by search, as SPOT does for trees
Leaf value	SPOT Theorem 1: leaf mean	mean as start, then search (rows do not share u_t, Σ_t)
Split choice	SPOT eq. 5: lowest summed SPO loss	the same, plus confirmation by the path Sharpe ratio (your addition)
Growth	SPOT: recursive, leaves independent; also an optimal-tree MILP	best-first, because the path couples the leaves
Pruning, forests	SPOT: validation pruning, SPO forests	not implemented
Protocol	both Wang–Hasuike articles: monthly rebalancing, rolling retraining, 3-month validation window, hyper-parameter tuning (Optuna; “Bayesian” in DIR)	weekly rows, single split, no tuning
Stabilisers	DIR: clipping ± 0.1 , rescaling, partial adjustment	bound $b = 0.1$ inside the search only
Metrics	W–H SPO article: return, volatility, Sharpe, Sortino, max drawdown	all except Sortino
DSPO	ranking of 4,000 stocks by a neural network	not used

Table 10: Where the code follows the articles and where it departs from them.